TSwap Protocol Audit Report

Version 1.0

Cyfrin.io

April 19, 2026

TSwap Protocol Audit Report

Cyfrin.io

April 19, 2026

Prepared by: vridhib

Lead Auditors: vridhib

Table of Contents

Protocol Summary

TSwap is a decentralized exchange protocol built around a constant product automated market maker (AMM). Users can swap between two any ERC20 token and WETH, provide liquidity by depositing both tokens in equal ratio, and withdraw their proportional share of the pool's reserves. Liquidity providers earn a 0.3% fee on all swaps, which is reinvested into the pool, increasing the value of their LP tokens over time. The protocol includes helper functions to simplify common actions: `deposit` for adding liquidity, `swapExactInput` and `swapExactOutput` for executing trades with slippage protection, and `sellPoolTokens` for easily exiting a liquidity position. The pool's core invariant is `x * y = k`, where `x` and `y` represent the reserves of the two tokens and `k` remains constant during swaps.

Disclaimer

The vridhib team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

- e643a8d4c2c802490976b538dd009b351b1c8dda

Scope

- ./src/PoolFactory.sol
- ./src/TSwapPool.sol

```
- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum
- Tokens: Any ERC20 token
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

The TSwap protocol was audited over a three-day period by one security researcher using a combination of manual review, static analysis (Slither), and fuzzing/invariant testing (Foundry). The audit focused on core swap mechanics, liquidity provision, fee accounting, and access controls as defined in the protocol summary.

Overall, the protocol suffers from multiple high-severity security flaws that make it unsuitable for production use without significant remediation. The most critical issues center on the `_swap` function, which breaks the constant product invariant by rewarding users with a free token every ten swaps—silently draining liquidity provider funds over time. Additionally, a mathematical error in `getInputAmountBasedOnOutput` causes users to be overcharged by approximately ten times the fair input amount for exact-output swaps. Compounding these issues, the `swapExactOutput` function lacks slippage protection (`maxInputAmount`), exposing users to unlimited losses from front-running, and the `sellPoolTokens` helper function incorrectly routes swaps, resulting in unpredictable token expenditures.

The findings are summarized as follows:

- **4 High-Severity** issues:
  - Invariant-breaking bonus token reward in `_swap`
  - 10 times overcharge for swappers due to incorrect fee denominator (`10000` instead of `1000`)
  - Missing slippage protection in `swapExactOutput`
  - Wrong swap direction in `sellPoolTokens`
- **2 Medium-Severity** issues:
  - Non-standard ERC20 tokens (fee-on-transfer, rebase, ERC777) break protocol invariants
  - Missing deadline in `deposit`
- **2 Low-Severity** issues:
  - Emitted event uses out-of-order arguments
  - Incorrect return value assignment for `swapExactInput`

- **9 Informational** issues: magic numbers, missing zero-address checks, missing or incomplete NatSpec, dead code, unused error, wrong function visibility, etc.

Due to the severity and number of findings, the protocol should not be deployed in its current state. It is strongly recommended to remove the bonus token logic from `_swap`, correct the fee denominator in `getInputAmountBasedOnOutput`, add a `maxInputAmount` parameter to `swapExactOutput`, and refactor `sellPoolTokens` to use `swapExactInput`. A follow-up audit is strongly advised after implementing the recommended changes.

Issues Found

Severity	Number of Issues Found
High	4
Medium	2
Low	2
Info	9
Total	17

Findings

High

[H-1] Incorrect Fee Calculation in `TSwapPool::getInputAmountBasedOnOuput` Results in Significant Fees for Users

Description:

The `getInputAmountBasedOnOuput` function is intended to calculate the amount of tokens a user should deposit given an amount of output tokens. However, the function currently miscalculates the resulting input amount. When calculating the fee, the function scales the amount by 10000 instead of 1000.

Impact:

The protocol takes more fees than expected from users, which will cause high user turnover.

Proof of Concept:

Insert the following test in `TSwapPool.t.sol`.

Proof of Code

```
    function testGetInputAmountBasedOnOutputChargesHighFees() public {
        uint256 lpAmount = 100e18;
        uint256 swapAmount = 10e18;

        // Fund user with ample poolToken
        poolToken.mint(user, 1000e18);

        // LP adds liquidity
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), lpAmount);
        poolToken.approve(address(pool), lpAmount);
        pool.deposit(lpAmount, lpAmount, lpAmount, uint64(block.
            timestamp));
        vm.stopPrank();

        // User calls `swapExactOutput` (which uses incorrect math)
        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        uint256 startingInputBalance = poolToken.balanceOf(user);
        uint256 startingOutputBalance = weth.balanceOf(user);
        pool.swapExactOutput(poolToken, weth, swapAmount, uint64(block.
            timestamp));

        // Fair expected input (correct fee math) vs unfair actual
            input (incorrect fee math)
        uint256 expectedRequiredInput = ((lpAmount * swapAmount) *
            1000) / ((lpAmount - swapAmount) * 997);
        uint256 actualRequiredInput = startingInputBalance - poolToken.
            balanceOf(user);

        // Check the output balance was as expected
        uint256 endingOutputBalance = startingOutputBalance +
            swapAmount;

        // The buggy contract uses 10000 instead of 1000, causing a ~10
            x overcharge.
        // Therefore actual input = ~10 * expected & output is as
            expected
        assertEq(endingOutputBalance, startingOutputBalance +
            swapAmount);
        assertApproxEqRel(actualRequiredInput, expectedRequiredInput *
            10, 0.01e18); // 1% tolerance
    }
```

Recommended Mitigation:

Change the 10000 value to 1000. Additionally, it is recommended to avoid using literals, and instead

store the 1000 value in a constant variable (e.g., FEE_DENOMINATOR). See I-7 for further explanation.

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
-       return ((inputReserves * outputAmount) * 10000) / ((
    outputReserves - outputAmount) * 997);
+       return ((inputReserves * outputAmount) * 1000) / ((
    outputReserves - outputAmount) * 997);

    }
```

[H-2] Lack of Slippage Protection in `TSwapPool::swapExactOutput` Potentially Causes Users to Receive Less Tokens Than Expected

Description:

The `swapExactOutput` function does not include any means of slippage protection. This function's logic is similar to that of `TSwapPool::swapExactInput`, in which it specifies a `minOutputAmount` parameter.

Impact:

If market conditions change before the transaction processes, the user could get an unfavorable or capital inefficient swap.

Proof of Concept:

1. The price of WETH is 1,000 USDC
2. User calls `swapExactOutput` to receive 1 WETH
    1. inputToken = USDC
    2. outputToken = WETH
    3. outputAmount = 1
    4. deadline = 2 weeks
3. The function does not offer a `maxInputAmount` amount

4. As the transaction is pending in the mempool, the market conditions change in which there are stark price fluctuations, causing the exchange ratio to be 1 WETH to 10,000 USDC. That fluctuation is ten times more costly than what the user expected.
5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC.

Proof of Code

```
    function testSwapExactOutputMissingSlippageProtection() public {
        // Setup: Provide initial liquidity to the pool
        uint256 initialLiquidity = 100e18;
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), initialLiquidity);
        poolToken.approve(address(pool), initialLiquidity);
        pool.deposit(initialLiquidity, initialLiquidity,
            initialLiquidity, uint64(block.timestamp));
        vm.stopPrank();

        // User wants to receive exactly 10 WETH
        uint256 desiredOutput = 10e18;

        // Calculate fair input required under current reserves (no
            attack)
        uint256 fairInput = ((initialLiquidity * desiredOutput) * 1000)
             / ((initialLiquidity - desiredOutput) * 997);

        // Attacker frontruns the user by executing a large swap that
            drains WETH
        // This drastically increases the price of WETH in terms of
            poolToken
        address attacker = makeAddr("attacker");
        uint256 attackAmount = 50e18;
        poolToken.mint(attacker, 1000e18);
        vm.startPrank(attacker);
        poolToken.approve(address(pool), type(uint256).max);
        pool.swapExactInput(poolToken, attackAmount, weth, 0, uint64(
            block.timestamp));
        vm.stopPrank();

        // User's transaction executes but the required input is much
            higher
        vm.startPrank(user);
        poolToken.mint(user, 1000e18);
        poolToken.approve(address(pool), type(uint256).max);
        uint256 startingBalance = poolToken.balanceOf(user);

        // Transactions succeeds despite the terrible exchange rate
        pool.swapExactOutput(poolToken, weth, desiredOutput, uint64(
            block.timestamp));
        uint256 actualInput = startingBalance - poolToken.balanceOf(
```

```
            user);
35          vm.stopPrank();
36
37          // Calculate what a reasonable maxInputAmount would have been
                (~5%)
38          uint256 reasonableMaxInput = (fairInput * 105) / 100;
39
40          // The actual input paid is far beyond any acceptable slippage
                tolerance
41          assertGt(actualInput, reasonableMaxInput, "User paid more than
                5% slippage");
42
43          // User paid approximately 20x the fair price due to missing
                protection
44          // This over-charge is exacerbated by the math error (10000 vs
                1000)
45          console.log("Fair input (no attack):     ", fairInput);  //
                ~11.1
46          console.log("Actual input paid:           ", actualInput);  //
                ~265.2
47          console.log("Reasonable max input (5%):   ", reasonableMaxInput
                ); // ~11.7
48      }
```

Recommended Mitigation:

The `swapExactOutput` should specify a `maxInputAmount` as such:

```
1  +    error TSwapPool__InputTooHigh(uint256 actual, uint256 max);
2
3       function swapExactOutput(
4           IERC20 inputToken,
5           IERC20 outputToken,
6           uint256 outputAmount,
7  +        uint256 maxInputAmount,
8           uint64 deadline
9       )
10          public
11          revertIfZero(outputAmount)
12          revertIfDeadlinePassed(deadline)
13          returns (uint256 inputAmount)
14      {
15          uint256 inputReserves = inputToken.balanceOf(address(this));
16          uint256 outputReserves = outputToken.balanceOf(address(this));
17
18          inputAmount = getInputAmountBasedOnOutput(
19              outputAmount,
20              inputReserves,
21              outputReserves
22          );
23
```

```
24 +         if (inputAmount > maxInputAmount) {
25 +             revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount)
       ;
26 +         }
27
28           _swap(inputToken, inputAmount, outputToken, outputAmount);
29       }
```

[H-3] `TSwapPool::sellPoolTokens` Mismatches Input & Output Tokens, Resulting in Users Receiving Incorrect Token Amounts

Description:

The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users specify the amount of pool tokens they are willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount. This is due to the fact that the `swapExactOutput` function is called instead of the `swapExactInput` function. Users should be specifying the exact amount of input tokens, not output.

Impact:

Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

To isolate the `sellPoolTokens` logic error from the separate fee calculation bug (H-2), temporarily correct the `getInputAmountBasedOnOutput` function by changing `10000` to `1000` for the duration of this test.

```
1       function getInputAmountBasedOnOutput(...) ...
2       {
3           ...
4           return
5   -           ((inputReserves * outputAmount) * 10000) /
6   +           ((inputReserves * outputAmount) * 1000) /
7               ((outputReserves - outputAmount) * 997);
8       }
```

Insert the following test into `TSwapPool.t.sol`.

Proof of Code

```
1       function testSellPoolTokensUsesWrongSwapFunction() public {
2           // Provide initial liquidity
3           uint256 initialLiquidity = 100e18;
4           vm.startPrank(liquidityProvider);
5           weth.approve(address(pool), initialLiquidity);
```

```
        poolToken.approve(address(pool), initialLiquidity);
        pool.deposit(initialLiquidity, initialLiquidity,
            initialLiquidity, uint64(block.timestamp));
        vm.stopPrank();

        // User wants to sell exactly 10 pool tokens for WETH
        uint256 sellAmount = 10e18;
        poolToken.mint(user, 100e18);
        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        uint256 startingPoolTokenBalance = poolToken.balanceOf(user);
        uint256 startingWethBalance = weth.balanceOf(user);

        // User calls the `sellPoolTokens` function
        uint256 returnedValue = pool.sellPoolTokens(sellAmount);
        vm.stopPrank();

        uint256 poolTokensSpent = startingPoolTokenBalance - poolToken.
            balanceOf(user);
        uint256 wethReceived = weth.balanceOf(user) -
            startingWethBalance;

        // Assert that user spent an variable amount of pool tokens (
            not the fixed `sellAmount`)
        assertNotEq(poolTokensSpent, sellAmount);
        // Assert that the return value equals the pool tokens spent (
            not the WETH received)
        assertEq(returnedValue, poolTokensSpent);
        assertNotEq(returnedValue, wethReceived);
        // Assert that the WETH received is exactly `sellAmount` (
            treated as desired output)
        assertEq(wethReceived, sellAmount);
    }
```

Recommended Mitigation:

Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (i.e., `minWethToReceive` passed to `swapExactInput`).

```
    function sellPoolTokens(
        uint256 poolTokenAmount
+       uint256 minWethToReceive
    ) external returns (uint256 wethAmount) {
-       return swapExactOutput(i_poolToken, i_wethToken,
    poolTokenAmount, uint64(block.timestamp));
+       return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken
    , minWethToReceive, uint64(block.timestamp));

    }
```

Additionally, it might be prudent to add a `deadline` parameter to the function to enforce deadlines.

[H-4] `TSwapPool::_swap` Function Incentivizes Users With a Free Token Every 10 Swaps, Breaking the Invariant

Description:

The protocol follows a strict invariant of `x * y = k` for swaps. In which: - `x` = The balance of the pool token - `y` = The balance of WETH - `k` = The constant product of the two balances

This means that during a swap the product of the reserves must remain unchanged (ignoring fees, which are accounted for separately). However, the `_swap` function breaks this invariant by transferring an extra `1e18` of the output token to the caller every 10 swaps, without receiving any corresponding input token.

This direct withdrawal of tokens from the pool's reserves reduces the product `x * y`, meaning `k` decreases over time. As a result, the pool becomes progressively insolvent relative to its pricing curve, draining liquidity provider funds and eventually making swaps impossible.

The following block of code is responsible for the issue.

```
1          swap_count++;
2          if (swap_count >= SWAP_COUNT_MAX) {
3              swap_count = 0;
4              outputToken.safeTransfer(msg.sender, 1
                   _000_000_000_000_000_000);
5          }
```

Impact:

A user could maliciously drain the protocol of funds by conducting a high number of swaps and collecting the extra incentive provided by the protocol. In essence, the protocol's core invariant is broken.

Proof of Concept:

1. A user swaps tokens 10 times and collects the extra incentive of 1 extra token.
2. The user continues to swap tokens until all the protocol's funds are drained.

Proof of Code

Place the following test into `TSwapPool.t.sol`.

```
1      function testInvariantBroken() public {
2          uint256 lpAmount = 100e18;
3          uint256 userAmount = 100e18;
4          uint256 outputWeth = 1e17;
```

```
5
6           // LP provides liquidity
7           vm.startPrank(liquidityProvider);
8           weth.approve(address(pool), lpAmount);
9           poolToken.approve(address(pool), lpAmount);
10          pool.deposit(lpAmount, lpAmount, lpAmount, uint64(block.
                timestamp));
11          vm.stopPrank();
12
13          // Fund the user
14          vm.startPrank(user);
15          poolToken.approve(address(pool), type(uint256).max);
16          poolToken.mint(user, userAmount);
17
18          // The user swaps tokens 10 times
19          for (uint256 i = 0; i < 9; i++) {
20              pool.swapExactOutput(poolToken, weth, outputWeth, uint64(
                    block.timestamp));
21          }
22          int256 startingY = int256(weth.balanceOf(address(pool)));
23          int256 expectedDeltaY = int256(-1) * int256(outputWeth);
24          pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
                timestamp));
25          vm.stopPrank();
26
27          // The invariant is broken with the free token every 10 swaps
28          uint256 endingY = weth.balanceOf(address(pool));
29          int256 actualDeltaY = int256(endingY) - int256(startingY);
30          assertEq(actualDeltaY, expectedDeltaY);
31      }
```

Recommended Mitigation:

It is recommended to remove the extra incentive mechanism. If this incentive mechanism must be kept, the product change should be accounted for in the swapping protocol invariant (`x + y = k`). Alternatively, these incentive tokens could be set aside in the same manner as fees are.

Medium

[M-1] `TSwapPool::deposit` is Missing Deadline, Removing Time Sensitivity For All Transactions

Description:

The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transaction to be completed by”–but this parameter is never utilized in the function

definition. As a consequence, operations that add liquidity to the pool might be executed at unexpected times (e.g., market conditions in which the deposit rate is unfavorable).

Impact:

Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept:

The `deadline` parameter is unused as shown in the terminal output of `forge build`.

```
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
   --> src/TSwapPool.sol:119:9:
    |
119 |         uint64 deadline
    |         ^^^^^^^^^^^^^^^
```

Recommended Mitigation:

Consider using the `revertIfDeadlinePassed` modifier in the `deposit` function as such:

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
        revertIfZero(wethToDeposit)
+       revertIfDeadlinePassed(deadline)
        returns (uint256 liquidityTokensToMint)
    {...}
```

[M-2] Non-Standard ERC20 Tokens (Rebase, Fee-On-Transfer, ERC777) Break the Constant Product Invariant

Description:

The `TSwapPool` contract is designed to work with arbitrary ERC20 tokens. However, it assumes that all tokens adhere strictly to the standard ERC20 behavior where: - A transfer of `amount` tokens results in exactly `amount` tokens being received by the recipient. - Token balances remain static except when explicitly transferred.

The following non-standard token types violate these assumptions.

Token Type	Non-Standard Behavior
Fee-on-Transfer Tokens	A percentage of the transferred amount is deducted as a fee. The recipient receives less than the `amount` specified.
Rebase Tokens	Token balances can change automatically over time (e.g., elastic supply tokens like Ampleforth).
ERC777 Tokens	The `transferFrom` function triggers a `tokensReceived` hook on the recipient contract, allowing reentrancy or arbitrary state changes mid-transfer.

Due to `TSwapPool` relying on the exact amounts received to maintain the invariant `x * y = k`, any discrepancy between the expected and actual token amounts will corrupt the pool's accounting. Over time, this can lead to incorrect pricing for swaps, loss of funds for liquidity providers, and pool insolvency where users cannot withdraw their full share.

Impact:

For fee-on-transfer tokens, the pool receives fewer tokens than the `deposit` or `swap` function accounts for. The invariant `x * y = k` is calculated using the expected amount instead of the actual received amount. This allows attackers to extract more value from the pool than they should, slowly draining liquidity. In the case of rebase tokens, automatic balance changes alter the pool's reserves without a corresponding swap or deposit, breaking the invariant and causing incorrect pricing. Liquidity providers may lose funds when withdrawing. With ERC777 tokens, the hook mechanism can be exploited for reentrancy attacks during swaps or deposits, potentially draining the pool entirely.

While the protocol owner can choose which tokens to support, the contract contains no safeguards to prevent the accidental or malicious addition of non-standard tokens. This is a medium severity issue because it requires specific token types to materialize security issues, but the impact is high (i.e., loss of funds, broken invariant) when such tokens are used.

Proof of Concept:

1. Deploy pool with non-standard FeeToken as input token and WETH as output token
2. User deposits 100 FeeToken and 100 WETH
3. User attempts a swap: expects to send 10 FeeToken and receive ~9 WETH
4. Actual received FeeToken by pool is 9.9 (after 1% fee)
5. Pool reserves are now misaligned with the invariant calculation
6. Subsequent swaps use incorrect pricing, draining value from LPs

Recommended Mitigation:

This is a potential mitigation to prevent against weird-ERC20 issues.

```
function _transferIn(IERC20 token, address from, uint256 amount)
    internal returns (uint256) {
     uint256 balanceBefore = token.balanceOf(address(this));
     token.safeTransferFrom(from, address(this), amount);
     uint256 balanceAfter = token.balanceOf(address(this));
     uint256 received = balanceAfter - balanceBefore;
     require(received == amount, "TSwapPool: Fee-on-transfer tokens not
        supported");
     return received;
}
```

Applying this check in `deposit` and `_swap` to ensure the contract's internal accounting matches the actual token balances. Additionally, clearly indicate in the protocol's user interface and documentation that only standard ERC20 tokens are supported. Alternatively, consider using a token whitelist controlled by the owner to restrict pool creation to audited, standard tokens. For ERC777 tokens, an additional reentrancy guard (OpenZeppelin's nonReentrant modifier) on all state-changing functions is recommended as defense-in-depth, even though the CEI pattern is followed.

Low

[L-1] `TSwapPool::LiquidityAdded` Event Has Arguments Out of Order

Description:

When the `LiquidityAdded` event is emitted in the `TSwapPool:_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the thrid argument position and the `wethToDeposit` value should go second.

Impact:

Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

Fix the ordering of the arguments in the `LiquidityAdded` event.

```
     function _addLiquidityMintAndTransfer(
         uint256 wethToDeposit,
         uint256 poolTokensToDeposit,
         uint256 liquidityTokensToMint
     ) private {
         _mint(msg.sender, liquidityTokensToMint);
-        emit LiquidityAdded(msg.sender, poolTokensToDeposit,
    wethToDeposit);
```

```
8  +       emit LiquidityAdded(msg.sender, wethToDeposit,
       poolTokensToDeposit);
9          ...
10     }
```

[L-2] Default Value Returned by `TSwapPool::swapExactInput` Results in Incorrect Return Value Given

Description:

The `swapExactInput` function is expected to return the actual amount of bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact:

The return value will always be 0, giving incorrect information to the caller.

Proof of Concept:

Insert the following test into `TSwapPool.t.sol`.

Proof of Code

```
1      function testSwapExactInputReturnsZero() public {
2          // Add initial liquidity
3          uint256 initialLiquidity = 100e18;
4          vm.startPrank(liquidityProvider);
5          weth.approve(address(pool), initialLiquidity);
6          poolToken.approve(address(pool), initialLiquidity);
7          pool.deposit(initialLiquidity, initialLiquidity,
               initialLiquidity, uint64(block.timestamp));
8          vm.stopPrank();
9
10         // User performs a swap
11         uint256 inputAmount = 10e18;
12         poolToken.mint(user, inputAmount);
13         vm.startPrank(user);
14         poolToken.approve(address(pool), type(uint256).max);
15         uint256 startingWethBalance = weth.balanceOf(user);
16
17         // Record the returned output versus the actual output received
18         uint256 returnedOutput = pool.swapExactInput(poolToken,
               inputAmount, weth, 0, uint64(block.timestamp));
19         vm.stopPrank();
20         uint256 actualWethReceived = weth.balanceOf(user) -
               startingWethBalance;
21
```

```
22          // Assert there is a discrepancy between the actual output and
                the expected output
23          assertEq(returnedOutput, 0);
24          assertGt(actualWethReceived, 0);
25      }
```

Recommended Mitigation:

```
1       function swapExactInput(
2           IERC20 inputToken,
3           uint256 inputAmount,
4           IERC20 outputToken,
5           uint256 minOutputAmount,
6           uint64 deadline
7       )
8           public
9           revertIfZero(inputAmount)
10          revertIfDeadlinePassed(deadline)
11          returns (uint256 output)
12
13      {
14          uint256 inputReserves = inputToken.balanceOf(address(this));
15          uint256 outputReserves = outputToken.balanceOf(address(this));
16
17  +       uint256 output = getOutputAmountBasedOnInput(
18  -       uint256 outputAmount = getOutputAmountBasedOnInput(
19              inputAmount,
20              inputReserves,
21              outputReserves
22          );
23
24  +       if (output < minOutputAmount) {
25  +           revert TSwapPool__OutputTooLow(output, minOutputAmount);
26          }
27  -       if (outputAmount < minOutputAmount) {
28  -           revert TSwapPool__OutputTooLow(outputAmount,
        minOutputAmount);
29          }
30
31  +       _swap(inputToken, inputAmount, outputToken, output);
32  -       _swap(inputToken, inputAmount, outputToken, outputToken);
33
34      }
```

Informational

[I-1] Unused Custom Error `PoolFactory::PoolFactory__PoolDoesNotExist` Wastes Deployment Gas

Description:

Remove the unutilized custom error in the `PoolFactory` contract to save gas upon deployment.

Recommended Mitigation:

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Missing Zero-Address Check for `PoolFactory` Constructor Parameter

Description:

Add a check for a zero-address for the `PoolFactory` constructor `wethToken` parameter.

Recommended Mitigation:

Add a check for a potential `address(0)` argument and a custom error to go along with it.

```
+ error PoolFactory__CannotBeZeroAddress();
  .
  .
  .
  constructor(address wethToken) {
+       if (wethToken == address(0)) revert PoolFactory__CannotBeZeroAddress();
        i_wethToken = wethToken;
  }
```

[I-3] `PoolFactor::createPool` Should Use `.symbol()` Instead of `.name()` For Semantic Accuracy

Description:

Incorrect usage of the ERC20 `.name()` method results in an inaccurate `TSwapPool` liquidity token symbol. For semantic accuracy use the `.symbol()` ERC20 method instead.

Recommended Mitigation:

In the `liquidityTokenSymbol` variable definition use `.symbol()` instead of `.name()`.

```
    function createPool(address tokenAddress) external returns (address
        ) {
        if (s_pools[tokenAddress] != address(0)) {
            revert PoolFactory__PoolAlreadyExists(tokenAddress);
        }

        string memory liquidityTokenName = string.concat("T-Swap ",
            IERC20(tokenAddress).name());
-       string memory liquidityTokenSymbol = string.concat("ts", IERC20
    (tokenAddress).name());
+       string memory liquidityTokenSymbol = string.concat("ts", IERC20
    (tokenAddress).symbol());
        TSwapPool tPool = new TSwapPool(tokenAddress, i_wethToken,
            liquidityTokenName, liquidityTokenSymbol);
        s_pools[tokenAddress] = address(tPool);
        s_tokens[address(tPool)] = tokenAddress;
        emit PoolCreated(tokenAddress, address(tPool));
        return address(tPool);
    }
```

[I-4] Missing Zero-Address Check for `TSwapPool` Constructor Parameters

Description:

Add a check for zero-addresses for the `TSwapPool` constructor `poolToken` and `wethToken` parameters.

Recommended Mitigation:

Add a check for potential `address(0)` arguments and a custom error to go along with it.

```
+  error TSwapPool__CannotBeZeroAddress();
    .
    .
    .
    constructor(
        address poolToken,
        address wethToken,
        string memory liquidityTokenName,
        string memory liquidityTokenSymbol
    ) ERC20(liquidityTokenName, liquidityTokenSymbol) {
+       if (poolToken == address(0) || wethToken == address(0)) revert
    TSwapPool__CannotBeZeroAddress();
        i_wethToken = IERC20(wethToken);
        i_poolToken = IERC20(poolToken);
    }
```

[I-5] `TSwapPool::MINIMUM_WETH_LIQUIDITY` is a Constant & Not Required

Description:

The `MINIMUM_WETH_LIQUIDITY` variable is a constant variable that is available for the user to query. Adding it to the event is unneccessary.

Recommended Mitigation:

Remove the unneccessary event parameter.

```
- error TSwapPool__WethDepositAmountTooLow(uint256 minimumWethDeposit, uint256 wethToDeposit);
+ error TSwapPool__WethDepositAmountTooLow(uint256 wethToDeposit);
```

[I-6] `TSwapPool::deposit` Contains Dead Code, Which Wastes Gas

Description:

Using dead code can waste deployment gas and result in liquidity providers spending extra gas whenever they call the `deposit` function.

Recommended Mitigation:

Remove the unused variable.

```
function deposit(...) external revertIfZero(wethToDeposit) returns (uint256 liquidityTokensToMint) {
    ...
    if (totalLiquidityTokenSupply() > 0) {
        uint256 wethReserves = i_wethToken.balanceOf(address(this));
-       uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
    ...
```

[I-7] Use Constant Variables Instead of Literals

Description:

The contract uses literal numeric values (`1000`, `997`, `1e18`) in multiple places without descriptive names. This reduces code readability and increases the risk of errors.

Note that `getInputAmountBasedOnOutput` also contains fee literals, but those are addressed in a separate high-severity finding (H-1) that corrects a mathematical error.

Recommended Mitigation:

Use descriptive constant variables instead of literals or “magic numbers”.

```
+    uint256 private constant FEE_DENOMINATOR = 1000;
+    uint256 private constant FEE_NUMERATOR = 997;
+    uint256 private constant ONE_UNIT = 1e18;

    function getOutputAmountBasedOnInput(...) ... {
+       uint256 inputAmountMinusFee = inputAmount * FEE_NUMERATOR;
-       uint256 inputAmountMinusFee = inputAmount * 997;
        uint256 numerator = inputAmountMinusFee * outputReserves;
+       uint256 denominator = (inputReserves * FEE_DENOMINATOR) +
    inputAmountMinusFee;
-       uint256 denominator = (inputReserves * 1000) +
    inputAmountMinusFee;
        return numerator / denominator;
    }

    function _swap(...) private {
        ...
        swap_count++;
        if (swap_count >= SWAP_COUNT_MAX) {
            swap_count = 0;
+            outputToken.safeTransfer(msg.sender, ONE_UNIT);
-            outputToken.safeTransfer(msg.sender, 1
    _000_000_000_000_000_000);
        }
        ...
    }

    function getPriceOfOneWethInPoolTokens() external view returns (
       uint256) {
        return
            getOutputAmountBasedOnInput(
+               ONE_UNIT,
-               1e18,
                i_wethToken.balanceOf(address(this)),
                i_poolToken.balanceOf(address(this))
            );
    }

    function getPriceOfOnePoolTokenInWeth() external view returns (
       uint256) {
        return
            getOutputAmountBasedOnInput(
+               ONE_UNIT,
-               1e18,
                i_poolToken.balanceOf(address(this)),
                i_wethToken.balanceOf(address(this))
            );
    }
```

[I-8] Public Function Not Used Internally

Description:

If a function is marked public but is not used internally, consider marking it as `external` to save gas.

Recommended Mitigation:

Change the `swapExactInput` function's visibility from **`public`** to `external`.

```
    function swapExactInput(
        IERC20 inputToken,
        uint256 inputAmount,
        IERC20 outputToken,
        uint256 minOutputAmount,
        uint64 deadline
    )
-       public
+       external
        revertIfZero(inputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 output)
    {...}
```

[I-9] Missing NatSpec Documentation in `TSwapPool::swapExactOutput` for `deadline`

Description:

The `swapExactOutput` function is missing NatSpec documentation for the `deadline` parameter.

Recommended Mitigation:

Add the following line to the function's NatSpec.

```
    /*
     * @notice figures out how much you need to input based on how much
     * output you want to receive.
     *
     * Example: You say "I want 10 output WETH, and my input is DAI"
     * The function will figure out how much DAI you need to input to
        get 10 WETH
     * And then execute the swap
     * @param inputToken ERC20 token to pull from caller
     * @param outputToken ERC20 token to send to caller
     * @param outputAmount The exact amount of tokens to send to caller
+    * @param deadline The deadline for the transaction to be completed
    by
```

```
12
13       */
14      function swapExactOutput(
15          IERC20 inputToken,
16          IERC20 outputToken,
17          uint256 outputAmount,
18          uint64 deadline
19      )
```